Anomaly detection using Fast Fourier Transform (FFT)

Last amended: 24th Oct 2026
#My folder: C:\Users\ashok\OneDrive\Documents\anomaly_detection\FFT\sensor_data\
Ref: Knime article here
Corresponding pdf file is also in the folder
GitHub place: Under Clustering and also hadoop repo
Virtual Machine: ubuntu_database under Documents/
#

Contents

Python conda environment: 2
KNIME help: 2
Fourier Transform: 2
Experiments on Colab: 5
What if the signals are aperiodic? 7
Fourier Series vs Discrete Fourier Transform 7
How an aperiodic signal is transformed 8
What is Fast Fourier Transform? 9
How is FFT used for anomaly detection? Give brief numerical example? 9
Supervised vs Unsupervised learning 10
Sensor data 10
Problem: 10
How does FFT help: 10
Model building 11
Testing Bad Data 11
Knime Workflow: 13
Workflow in parts: 14
Data node-by-node 16
Why is each data point matched to '*corresponding*' frequency? 18
Would freq of normal and bad data be the same? 24
Why do I always get discrete frequency output in FFT? 25
Finance data 26

Python conda environment:

On every machine create conda python environment as below. Here 5.12 is the version of my KNIME.

```
conda create --name knime_env -c knime -c conda-forge knime-python-scripting=5.12 python=3.11 -y
```

Preferences-> Python->conda and select your environment.

If your environment does not appear here, then, open *Preferences->Conda* (see below). And write the path to your Anaconda3 folder. (Get it from *conda env list*). Once this is set then, you will find the newly created python env listed in *Preferences-> Python->conda*.

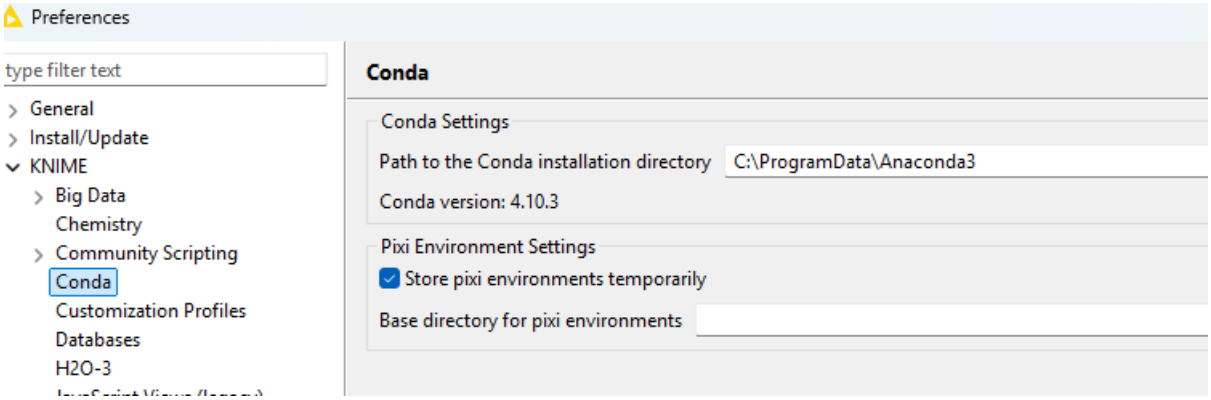


Figure 1: Write the path to Anaconda folder here.

KNIME help:

If KNIME does not start in Classic Interface, try these steps:

a. From where KNIME is installed:

```
rm -rf /home/ashok/knime_5.12.0/configuration/org.eclipse.osgi
rm -rf /home/ashok/knime_5.12.0/configuration/org.eclipse.update
```

b.

```
rm -rf ~/.knime
```

b. In knime.ini, write the following line at the bottom line:

```
-Dknime.ui.modern=false
```

e. Start knime as:

```
export GDK_BACKEND=x11
/home/ashok/knime_5.12.0/knime
```

Fourier Transform:

The **Fourier transform** translates a signal from the **time domain** (how it changes over time) into the **frequency domain** (the individual frequencies that make it up).

A classic analogy is thinking of a musical chord: the original sound wave is the time domain, while the individual musical notes (frequencies) and their volumes make up the frequency domain.

🔄 Time Domain vs. Frequency Domain

Feature	Time Domain (f(t))	Frequency Domain (F(ω))
What it shows	Amplitudes shifting over chronological time.	The strength (magnitude) and phase of each frequency pitch.
Visual representation	A standard waveform graph (e.g., an audio wave).	A spectrum graph showing spikes at specific frequencies.
Common Use	Recording audio, viewing voltage drops over time.	Equalizing audio, filtering noise, compression.

All the following signals are periodic.

Both Signal1 and Signal2 have the same frequency. But there is phase shift.

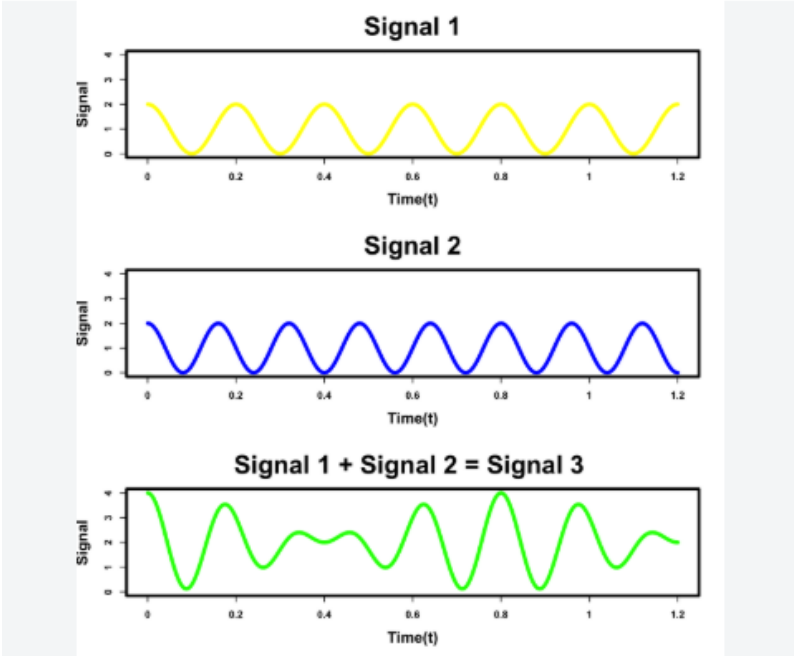


Figure 2: Fourier Transform would break Signal3 into two sinusoidal waves, each having one fixed frequency. Signal 3 is sum of just two signals: Signal 1 and Signal2.

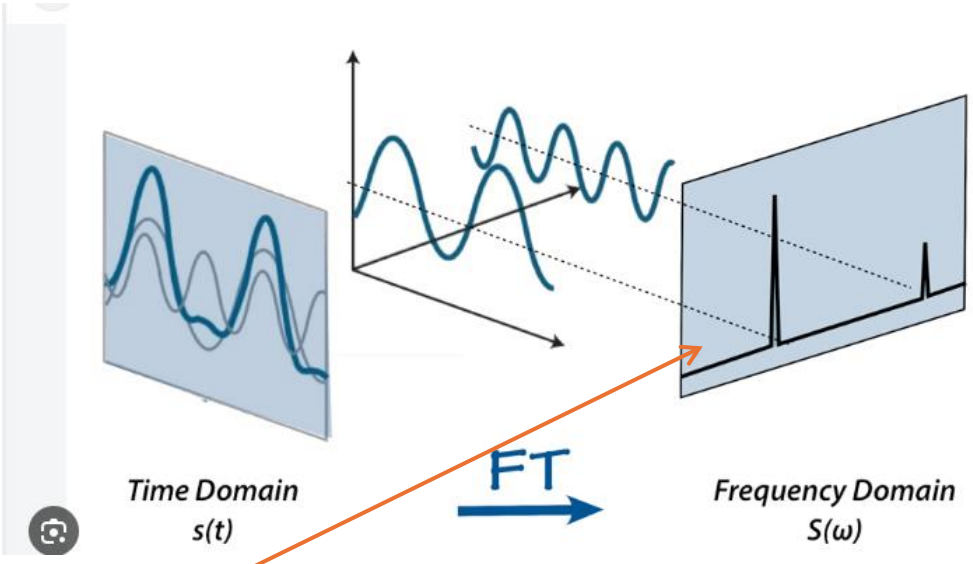


Figure 3 : The rightmost diagram is Frequency vs Amplitude (or Power)In the X-axis of frequency domain is NOT time but frequency.

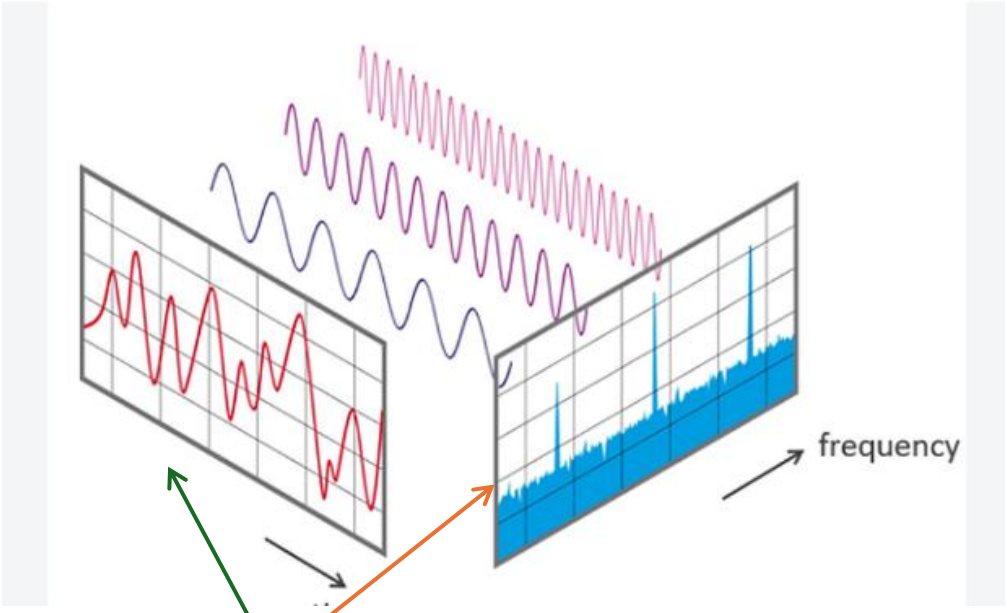


Figure 4: Interpret this as follows: This signal is sum of three sine-waves, each one of fixed frequency. But there may also be multiple frequencies of very low-amplitude also as per the hazy bottom.

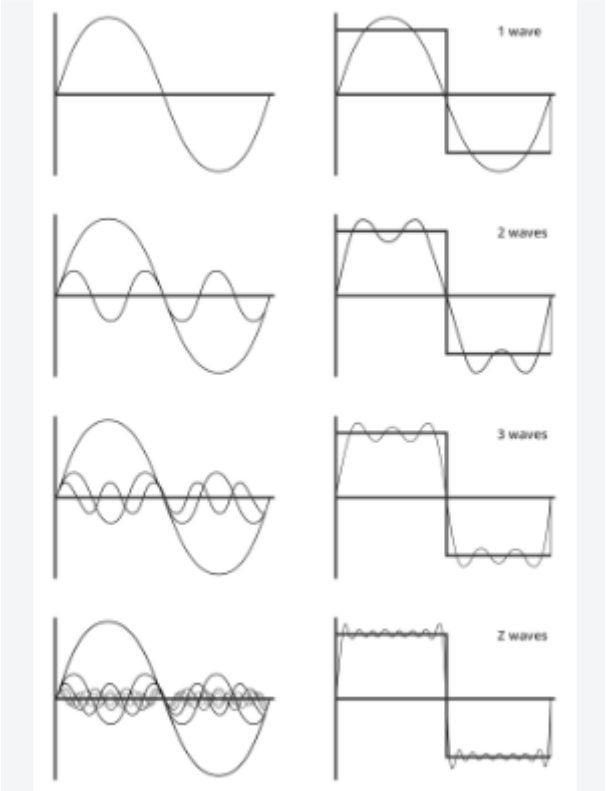


Figure 5: A square wave on the right being successively represented by a series of sine waves but of fixed frequencies.

Experiments on Colab:

If possible, create experiments on Colab/Claude Desktop or Google AI drawing up phase shifted sine waves and adding them up. Here are my prompts in Google Colab:

Starter prompt:

```
Generate two phase-shifted sinewaves of equal amplitude and add them up to
create a third sine wave.
```

Then, I changed wave phases in the code. The overall wave is still a sinewave.

Next prompt

```
Add a third sinewave to it such that overall wave does not look like a
sinewave but a distorted wave.
```

Next prompt:

```
How would you represent the above in a frequency domain. Draw this in the
next cell

OR

Plot the Fast Fourier Transform (FFT) of the final distorted wave
```

Next prompt:

```
But it has only two frequencies as against three we added? Why?
```

Answer:

```
That's a great observation! The reason you're seeing only two dominant
frequencies in the FFT plot, even though we generated three sine waves, is
due to how frequencies combine.
```

- The first two sine waves (sine_wave_1 and sine_wave_2) both have a frequency of **1.0 Hz**. When two sine waves of the same frequency are added together, they combine into a single resultant sine wave that *still has that same fundamental frequency* (1.0 Hz), but with a new amplitude and phase.
- The third sine wave (distorting_sine_wave) has a frequency of **3.0 Hz**.

So, when we sum all three, the resulting distorted_wave effectively contains two distinct frequency components: one at **1.0 Hz** (from the first two waves combining) and another at **3.0 Hz** (from the distorting wave). The FFT accurately reflects these two underlying frequency components in the final signal.

Next Prompt:

Can you create three aperiodic signals and add them up?

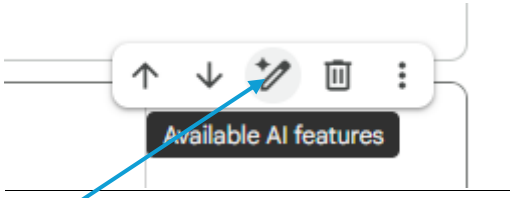
Answer:

Certainly! I will generate three different types of aperiodic signals, add them together, and then visualize both the individual signals and their combined output, including its FFT.

Next Prompt

How would you represent the combined graph in frequency domain?

Next Prompt:



In Google Colab, if you want to ask a question pertaining to any cell before, click inside that cell and then click the *Magic Wand* and click 'Transform Code to to take you to chat box for that cell.

Now ask:

Just write the FFT's output as a set of data points in a table.

Answer

Certainly! Here's the Fast Fourier Transform (FFT) output of the summed aperiodic wave presented as a table, showing the frequency, magnitude, and phase angle.

The Fast Fourier Transform (FFT) output for the summed aperiodic wave has been successfully generated and presented in a tabular format. Here are the first few rows of the FFT data:

Frequency (Hz)	Magnitude	Phase (radians)
0.0	124.910263	-0.000000
0.2	82.704781	2.880392
0.4	75.990989	-0.163212
0.6	47.161324	-3.028479

Frequency (Hz)	Magnitude	Phase (radians)
0.8	40.108188	-0.056551

This table shows the frequency, magnitude, and phase for each component of the FFT. If you'd like to see more of the data, let me know!

What if the signals are aperiodic?

The short answer is: **The Fourier transform is actually specifically designed for non-periodic signals.**
If a signal is **periodic** (repeats forever), we use **Fourier Series**, which breaks the signal down into a discrete set of specific, isolated frequencies. But if a signal is **non-periodic** (a transient pulse, a one-time event, or random noise), we must use the **Fourier Transform**, which maps the signal across a **continuous spectrum** of frequencies.
Here is how the transition works conceptually and mathematically:

From Periodic to Non-Periodic
To understand a non-periodic signal, mathematicians imagine a periodic signal and stretch its repetition period T out to **infinity.**

- **Fourier Series (Periodic):** Because the signal repeats regularly, the frequencies involved are perfectly spaced harmonics (e.g., 100Hz, 200Hz, 300Hz). The spectrum looks like a series of **isolated spikes** (lines).
- **Fourier Transform (Non-Periodic):** As the period approaches infinity, the gap between these harmonic frequencies shrinks to zero. Instead of isolated spikes, the spikes blend together perfectly to form a **smooth, continuous curve**.

Fourier Series vs Discrete Fourier Transform

The core difference between the **Fourier Series (FS)** and the **Discrete Fourier Transform (DFT)** is the type of signal they analyse: the Fourier Series breaks down continuous, periodic signals into infinite frequency components, whereas the Discrete Fourier Transform breaks down discrete, finite-length signals into a finite number of frequency bins (say after Nyquist sampling).
In short, the Fourier Series is a theoretical mathematical tool for continuous math, while the DFT is a computational tool designed for computers to process digitized data.

Key Comparison

Feature	Fourier Series (FS)	Discrete Fourier Transform (DFT)
Time Domain	**Continuous** (Infinite data points over a interval)	**Discrete** (Finite number of samples, N)
Signal Nature	Strictly **Periodic**	**Finite-length** (Implicitly treated as periodic)
Frequency Domain	**Discrete & Infinite** (Harmonics $1\omega_0$, $2\omega_0$, $3\omega_0$...)	**Discrete & Finite** (Exactly N frequency bins)
Mathematical Operation	**Integration** over one period	**Summation** over N points
Primary Use Case	Analytical math, physics, solving differential equations	Digital Signal Processing (DSP), audio, image files, coding

Understanding the Duality (Analogy)
Think of the **Fourier Series** like looking at a perfectly smooth, ongoing analog clock. The movement is continuous, and to break it down, you must calculate exactly how the individual planetary gears interact using smooth calculus integrals.
The **DFT** is like taking a digital video of that clock at 30 frames per second for just 10 seconds. You no longer have smooth data; you have a finite spreadsheet of snapshot numbers. A computer cannot process "infinity," so the DFT translates the math into simple addition and multiplication that hardware can easily solve.
How They Handle the Frequency Spectrum

- **Fourier Series:** Because the continuous signal repeats forever, its frequency spectrum consists of perfect, infinitely thin lines at exact harmonic intervals (f_0, $2f_0$, $3f_0$, etc.). It spans up to infinite frequency components. [1, 2]
- **DFT:** Because the data is sampled digitally, you can only look at a finite number of frequency points. Furthermore, due to sampling limits (the Nyquist frequency), the frequencies wrap around a finite circular structure rather than expanding to infinity.

*(Note: The **Fast Fourier Transform (FFT)** is not a different mathematical transform; it is simply a highly optimized algorithm used by computers to calculate the DFT much faster).* [1, 2]
Are you trying to **implement** one of these in code (like Python or MATLAB), or are you studying the **theoretical math** behind signal analysis? I can provide the **exact formulas** or code templates depending on what you need. [1, 2]

How an aperiodic signal is transformed

Query: So in a DFT first you digitize an aperiodic signal and then transform it into discrete frequency domain

Yes, you have the exact intuition, but with one critical nuance that often trips people up: **a computer mathematically treats that aperiodic signal as if it repeats forever.**
Here is the exact step-by-step breakdown of what happens under the hood when you compute a DFT:
1. Digitization (Sampling in Time)
You start with a real-world, continuous, aperiodic signal (like a 3-second audio recording of a dog barking).

- You sample it at a specific rate (\(f_{s}\)).
- This leaves you with a finite vector of \(N\) numbers: \([x_0, x_1, x_2, ..., x_{N-1}]\).

2. The "Periodic" Trap (Implicit Periodicity)
A computer cannot process a truly isolated, aperiodic event without assuming it is part of a larger pattern. The mathematical formula of the DFT implicitly forces your finite, \(N\)-point signal into a loop.
To the DFT algorithm, your 3-second dog bark doesn't just stop. The math acts as if that exact 3-second window **repeats over and over again, from negative infinity to positive infinity.**
3. Transformation (Discrete Frequencies)
Because the math forces the time-domain signal to look periodic, the resulting frequency domain becomes **discrete** (sampled into exact bins).

- You feed in \(N\) time samples.
- You get out exactly \(N\) frequency bins.
- These bins represent the discrete harmonic frequencies of that assumed periodic loop.

The Consequence: Spectral Leakage
Because the DFT forces an aperiodic signal to loop, the math expects the end of your signal (\(x_{N-1}\)) to splice perfectly back into the beginning (\(x_{0}\)).
If they don't match up smoothly (which rarely happens with aperiodic real-world data), you get a sharp, unnatural jump at the boundary. In the frequency domain, this sudden artificial jump creates "fake" high frequencies. This phenomenon is called **spectral leakage**, and it is why

engineers use **windowing functions** (like a Hann or Hamming window) to fade the edges of the signal to zero before running a DFT.

What is Fast Fourier Transform?

The **Fast Fourier Transform (FFT)** is not a new mathematical transform; it is simply a highly efficient **algorithm used to compute the Discrete Fourier Transform (DFT)**.
While the standard mathematical formula for the DFT is too slow for real-world computers to calculate in real time, the FFT uses clever mathematical shortcuts to compute the exact same result in a **fraction of the time**. It is widely considered one of the most important algorithms of the 20th century because it made digital signal processing practical.

⚡ Why the FFT is a Game Changer: Speed

To understand why the FFT is special, you have to look at its computational complexity (how much work a computer has to do as the signal gets longer). If your signal has **N** data points:

- **Standard DFT:** Has a complexity of **$O(N^2)$**. If you double the number of data points, the computer has to do four times as much work.
- **FFT Algorithm:** Has a complexity of **$O(N \log N)$**. It breaks the problem down recursively, drastically cutting down on repetitive calculations.

How is FFT used for anomaly detection? Give brief numerical example?

"Anomaly detection" refers to identifying deviations from normal patterns in signal data.

The FFT detects anomalies by converting a time-series signal into its frequency components to see if **unexpected frequencies appear** or if **expected frequencies suddenly drop or spike**.

While many defects (like a microscopic crack in a rotating motor or a subtle heartbeat irregularity) are nearly invisible in a messy timeline graph, they stick out in the frequency domain like a sore thumb.

🔍 The 3-Step Detection Process

1. **Establish a Baseline:** Run the FFT on a system operating normally. Note which frequencies are present and how strong they are (their "normal" amplitude envelope).
2. **Monitor Live Data:** Continuously take small time windows of live data and run an FFT on them.
3. **Threshold Checking:** Compare the live frequency spectrum against the baseline. If an unauthorized frequency spikes past a set threshold, flag it as an anomaly.

🔢 Brief Numerical Example

Imagine we are monitoring a **cooling fan** in a server rack. Under normal conditions, the fan spins at a steady speed, creating a predictable vibration wave.

1. Normal State (Baseline)

Suppose our live data gives us a clean wave, and running an FFT on 4 data samples yields the following frequency amplitudes:

- **0 Hz (DC offset):** 0.1
- **10 Hz (Fan rotation speed): 5.0** *(This is our expected, strong spike)*

- **20 Hz (Harmless minor vibration):** 0.2
- **30 Hz (High-frequency background noise):** 0.1

Rule set: If any frequency amplitude other than 10 Hz goes **above 1.5**, sound an alarm.

2. Abnormal State (The Anomaly)

A tiny piece of dust chips one of the fan blades. To the naked eye or a simple timeline sensor, the fan is still spinning and pulling power normally. However, the chipped blade causes a tiny, rapid wobble every time it spins.

We take a 4-sample snapshot of this new vibration wave and compute the FFT. The new amplitudes are:

- **0 Hz:** 0.1
- **10 Hz:** 4.8 *(Normal fan speed is slightly lower due to drag)*
- **20 Hz:** 0.3
- **30 Hz: 2.4** 🔴 **[ANOMALY DETECTED]**

💡 **The Verdict**

Because the **30 Hz frequency jumped from 0.1 to 2.4** (crossing our safety threshold of 1.5), the algorithm immediately flags a physical defect in the fan blade—long before the machine actually overheats or breaks down.

Supervised vs Unsupervised learning

Our sensor data is libelled. We know, in advance, which sensor readings are non-anomalous and which one are anomalous. Our approach, therefore, is that of Supervised learning. We first 'train' the model with non-anomalous data and then try to predict anomalous data. However, if our data is unlabelled then the approach would be first to calculate discrete FFTs and then use Isolation Forest to make predictions about anomalous data. This approach is used in Bank Transaction data set.

Sensor data

Once a machine is bad, it continues to emit bad sensor signals. So, this anomaly detection process is NOT to detect a point as anomalous BUT the whole batch as anomalous.

Problem:

We have a machine fitted with several sensors. We have time-wise data from each sensor when the machine had no problems. That is state is normal.

Suddenly, a noise is being emitted by a part of machine. Corresponding sensor is *AccelerometerX*. The problem is to predict if the data being emitted by *AccelerometerX* is anomalous.

We have two files:

a) `anomalous_machine_data.csv. AND`
b) `non_anomalous_machine_data.csv`

Data points are: `8971 AND 8193.`

How does FFT help:

FFT takes a fixed number of time-series points of a signal. Number of these points is a power of 2 i.e. 2^n. In our case, we take 512 time-points. FFT examines this complete time series of 512

points and tells us what all sine-wave frequencies and their amplitude would be able to reconstruct it.

We take many batches of these 512 samples. For each one them, we get amplitudes of sine-waves (of the same frequencies). Thus, for every single frequency, across the batches, we have different amplitude values. Mean of these values is taken as also variance.

Model building

Here is the process. FFT only accepts time-samples of size 2^n, so we first take a time sample of 4-points i.e. 2^2 points per sample. We feed into FFT, one batch at a time. As we have a total of fourteen such points, we break them into batches—and remaining two points are discarded. Let signal frequencies and their amplitudes for the three batches, as analysed by FFT, be as below:

frequency	amp1	amp2	amp3
0.2	2	2.1	1.9
0.4	1	1.2	0.8
0.6	3	2.8	3.1
0.8	0.9	1.1	0.8

Note that FFT does also output phase-shift information. But we are not using it but throwing it away. Why do I get discrete frequency output in FFT? Here in steps of 0.2. See below for an answer. Next, calculate mean & variance, row-wise or freq-wise. So now we have:

frequency	amp1	amp2	amp3	mean(amp)	var(amp)
0.2	2	2.1	1.9	2.00	0.0066667
0.4	1	1.3	0.8	1.03	0.0422222
0.6	3	2.8	3.1	2.97	0.0155556
0.8	0.9	1.1	0.8	0.93	0.0155556

This is our Model: For every frequency what is the mean amplitude and its variance

frequency	mean(amp)	var(amp)
0.2	2.00	0.0066667
0.4	1.03	0.0422222
0.6	2.97	0.0155556
0.8	0.93	0.0155556

Testing Bad Data

A data is bad, if and only if, frequency wise its amplitude is two-standard deviations away from the mean amplitude.

We now take (only) one sample of time-series (of 4-pts) about which we want to find, if it is anomalous or not. As before, our time-series data would have only four points. After FFT transformation our data frequency and amp are as below:

Our sample data:

frequency	ampx
0.2	3.4
0.4	2.1
0.6	1.8
0.8	1.5

We merge this data with our Model, frequency-wise:

frequency	ampx	mean(amp)	var(amp)
0.2	3.4	2.00	0.00666667
0.4	2.1	1.03	0.04222222
0.6	1.8	2.97	0.01555556
0.8	1.5	0.93	0.01555556

Next, we standardise our *ampx* column using row-wise (calculated) mean and variance, as below.

frequency	ampx	mean(amp)	var(amp)	stdamp
0.2	3.4	2.00	0.00666667	17.15
0.4	2.1	1.03	0.04222222	5.19
0.6	1.8	2.97	0.01555556	9.35
0.8	1.5	0.93	0.01555556	4.54

Take the overall mean of this '*stdamp*' column:

mean(stdamp)
9.06

The formula for calculating values in '*stdamp*' column is:

$$\text{abs}((\text{ampx} - \text{mean}(\text{amp}))/\text{sqrt}(\text{varamp}))$$

If this mean is >2, we conclude that this batch is anomalous.

Knime Workflow:

Knime workflow file is: '`Fourier Transform for Anamoly Detection.knwf.`' in the folder given above. Zoom it 300%. FFT component is NOT available in Knime. You will have to drag and drop it from Knime Hub or copy and paste from one of your workflows. **Knime needs python env.**

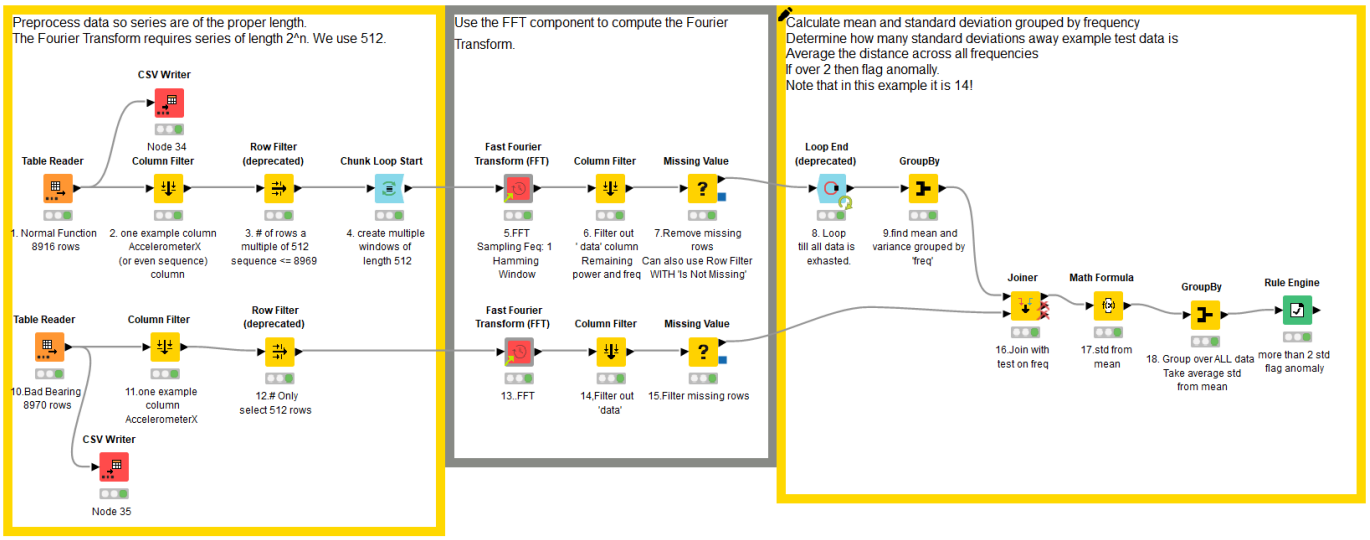
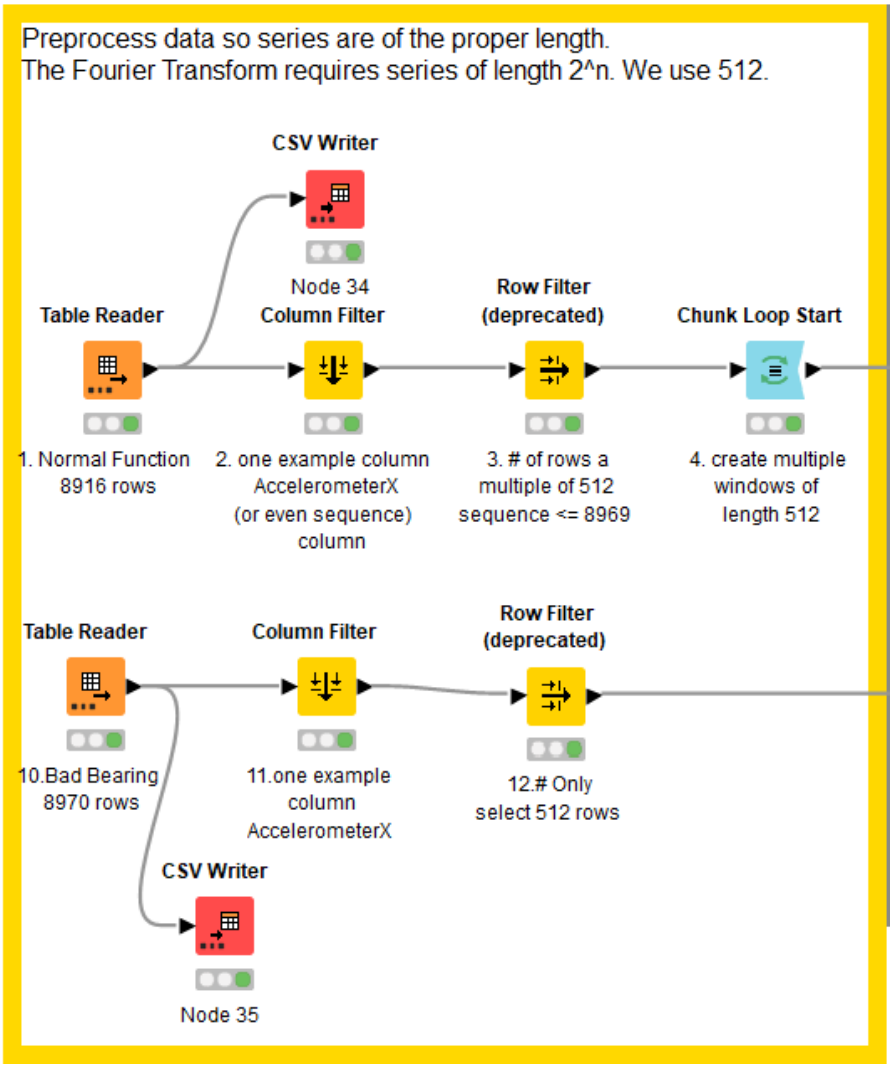
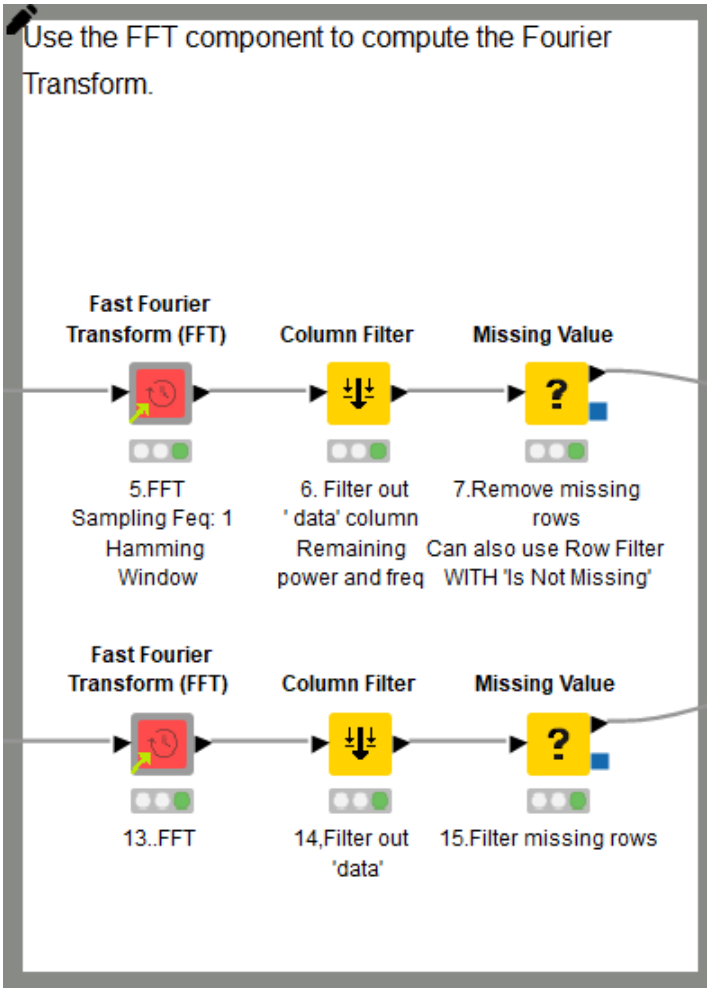
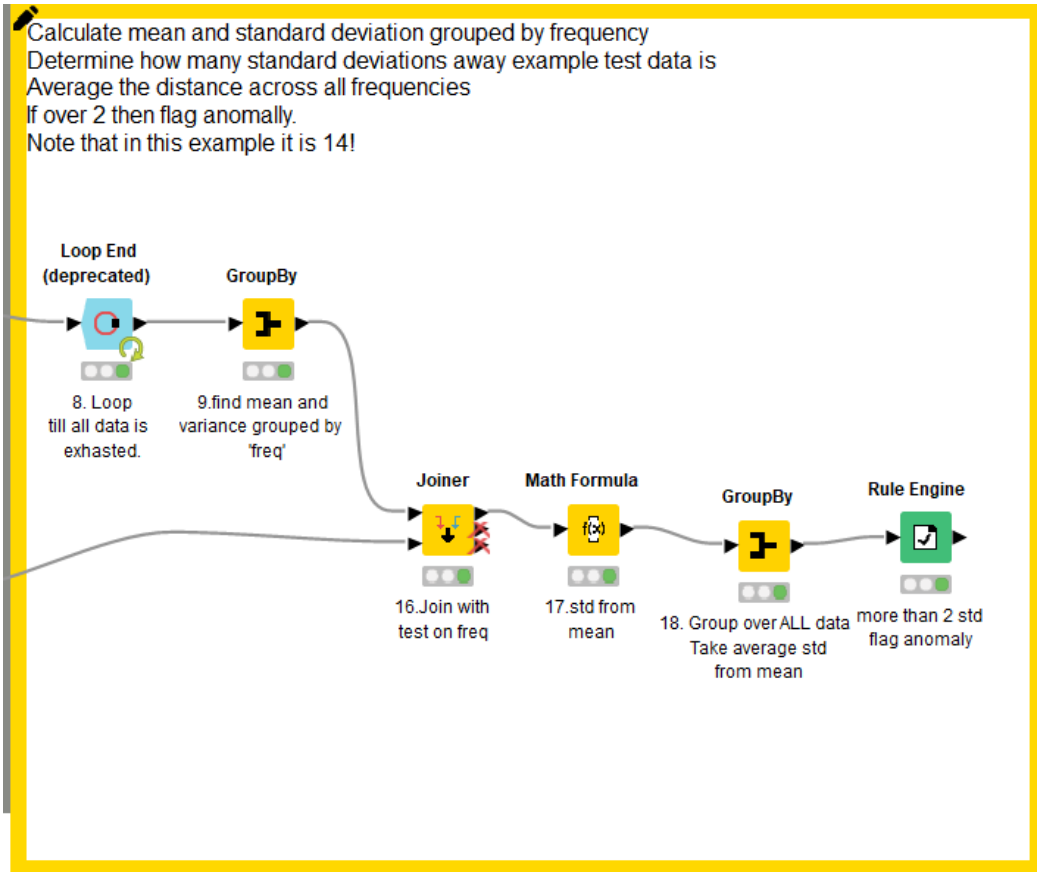


Figure 6: Drag and Drop FFT component from Knime Hub OR copy-paste from one of your workflows.

Workflow in parts:




Use the FFT component to compute the Fourier Transform.
Fast Fourier Transform (FFT)
Column Filter
Missing Value
5.FFT Sampling Feq: 1 Hamming Window
6. Filter out ' data' column Remaining power and freq
7.Remove missing rows Can also use Row Filter WITH 'Is Not Missing'
Fast Fourier Transform (FFT)
Column Filter
Missing Value
13..FFT
14,Filter out 'data'
15.Filter missing rows



Data node-by-node

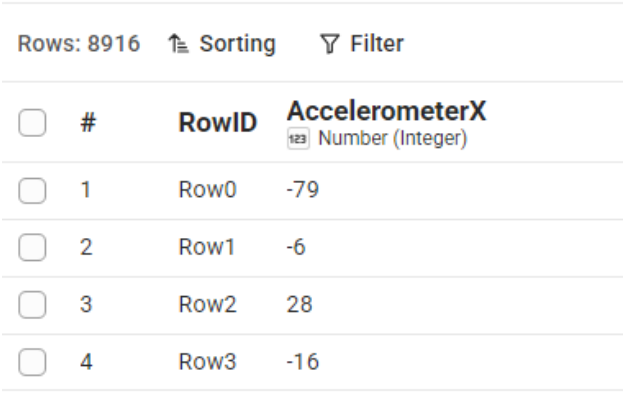
In our dataset we have a number of sensors. AccelerometerX is emitting noise and appears to be defective. Before the defect, the pattern of signal was as in is in Node1.

Node1: Original normal data at Node 1:

#	RowID	sequence	AccelerometerX	AccelerometerY	...	AccelerometerZ	GyroscopeX	GyroscopeY	GyroscopeZ
		Number (Integer)	Number (Integer)	Number (Integer)		Number (Integer)	Number (Integer)	Number (Integer)	Number (Integer)
1	Row0	0	-79	-977		25	-11	10	25
2	Row1	1	-6	-978		28	-12	13	16
3	Row2	2	28	-979		48	-12	11	17
4	Row3	3	-16	-981		23	-15	12	22
5	Row4	4	-48	-980		21	-11	11	20

Node2

Node2. Only one column (AccelerometerX) is being examined in this problem. We will find anomaly only through this column:

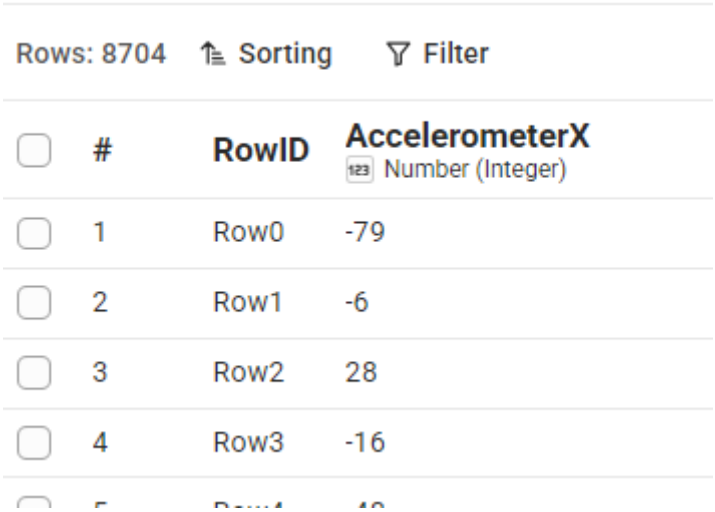


Rows: 8916 Sorting Filter

#	RowID	AccelerometerX (Number (Integer))
1	Row0	-79
2	Row1	-6
3	Row2	28
4	Row3	-16

Node3:

Reduce rows so that overall number is a multiple of 512 (some power of 2 ie 2^n). The number of data points in an FFT is chosen as 512 because it is a power of 2, which is a strict mathematical requirement for the most common and efficient version of the algorithm (the **Cooley-Tukey algorithm**). We will take 8970 rows.



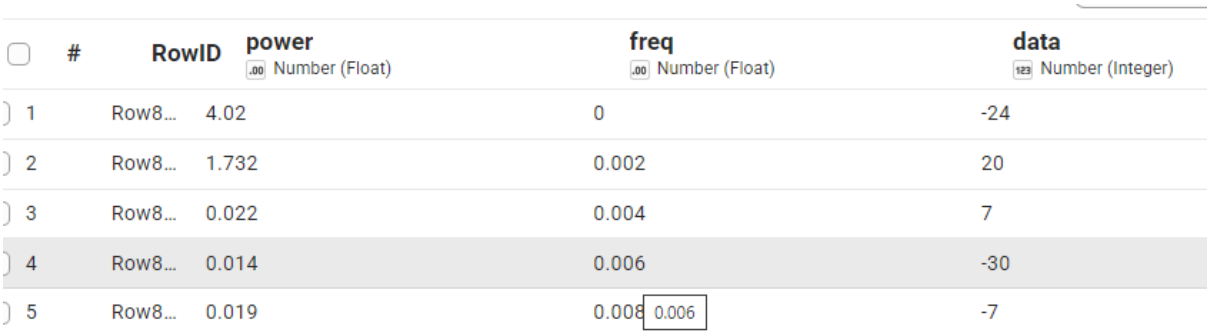
Rows: 8704 Sorting Filter

#	RowID	AccelerometerX (Number (Integer))
1	Row0	-79
2	Row1	-6
3	Row2	28
4	Row3	-16

Node4, takes chunk-by-chunk of 512 data points. Transforms this time-varying data to frequency domain per chunk.

Node5:

FFT node outputs as follows:



#	RowID	power (Number (Float))	freq (Number (Float))	data (Number (Integer))
1	Row8...	4.02	0	-24
2	Row8...	1.732	0.002	20
3	Row8...	0.022	0.004	7
4	Row8...	0.014	0.006	-30
5	Row8...	0.019	0.008	-7

Figure 7: The 'data' column is AccelerometerX data.

Why is each data point matched to '*corresponding*' frequency?

The table contains 'RowID', 'power', 'freq', and the raw 'data' (AccelerometerX data values). One is confused about why each raw data point seems to line up one-to-one with a specific frequency entry in the output table.

Each data point in the table isn't actually mapping to a single frequency. Instead, what you are looking at is a side-by-side **alignment of two entirely separate domains**—the time domain and the frequency domain—arranged into a single table for convenience.

When you run a Discrete Fourier Transform (or FFT), the algorithm takes an **entire block of time-domain data** (e.g., thousands of sequential points from your accelerometer) all at once and transforms them into an **entire block of frequency-domain data**.

Here is exactly why they appear side-by-side in your table and what that data layout actually means:

🧱 1. Equal Input and Output Sizes

A fundamental rule of the Discrete Fourier Transform is that **the number of inputs equals the number of outputs**.

- If you feed the FFT algorithm **N time-domain samples** (your data column),
- It will output exactly **N frequency-domain bins** (your freq and power columns).

Because the output has the exact same number of rows as the input, data processing software (like KNIME, Excel, or Python dataframes) will naturally display them side-by-side in the same row index (Row8192, Row8193, etc.).

🕰️ Time Domain vs. 🎶 Frequency Domain

The table merges two completely different scales that do not share the same "meaning" per row:

- **The data column reads Chronologically:** Row 1 (Row8192) is your accelerometer reading at Time = 0. Row 2 is the reading a fraction of a second later, Row 3 is after that, and so on.
- **The freq and power columns read Spectral Scale:** Row 1 is the power of the **0 Hz component** (the static gravity/DC offset of the whole signal chunk). Row 2 is the power at **0.002 Hz**, Row 3 is at **0.004 Hz**, and so forth.

Therefore, the value 20 in the data column of Row 2 **did not** produce the 1.732 power at 0.002 Hz. Rather, **every single number** in the data column combined together mathematically to generate the entire power spectrum column.

If you change just **one** value in the data column, it will alter the power values across **all** the rows in the frequency columns, because every frequency bin requires looking at the entire time signal to be calculated.

Node6

Filters out '*data*' point:

Rows: 512 | Columns: 2 Sorting Filter

#	RowID	power Number (Float)	freq Number (Float)
1	Row8...	4.02	0
2	Row8...	1.732	0.002
3	Row8...	0.022	0.004
4	Row8...	0.014	0.006
5	Row8...	0.019	0.008
6	Row8...	0.015	0.01
7	Row8...	0.02	0.012

Node 7

Remove missing rows. Number of rows is now reduced to 256 from 512.

Rows: 256 | Columns: 2 Sorting Filter

#	RowID	power Number (Float)	freq Number (Float)
1	Row8...	4.02	0
2	Row8...	1.732	0.002
3	Row8...	0.022	0.004
4	Row8...	0.014	0.006
5	Row8...	0.019	0.008
6	Row8...	0.015	0.01

Node8:

Loop until all data in batches of 512 is exhausted. Note also that in every batch, our starting frequency is 0.002 and ending frequency is: 0.248. In the following diagram, we have different values of frequency '*power*' or frequency amplitude for each one of the 16 interactions (i.e. batches). (16 X 512 = 8192).

Frequency gap is 0.002 i.e. frequencies are: 0.248, 0.246, 0.244....

Collected results (Table)

Rows: 4352 | Columns: 3 | Sorting | Filter

#	RowID	power (Number (Float))	freq (Number (Float))	Iteration (Number (Integer))
128	Row127#0	0.738	0.248	0
384	Row639#1	0.241	0.248	1
640	Row1151#2	0.277	0.248	2
896	Row1663#3	0.295	0.248	3
1152	Row2175#4	0.527	0.248	4
1408	Row2687#5	0.173	0.248	5
1664	Row3199#6	0.957	0.248	6
1920	Row3711#7	0.415	0.248	7
2176	Row4223#8	0.128	0.248	8
2432	Row4735#9	0.546	0.248	9
2688	Row5247#10	0.751	0.248	10
2944	Row5759#11	0.304	0.248	11
3200	Row6271#12	0.258	0.248	12
3456	Row6783#13	0.411	0.248	13
3712	Row7295#14	0.608	0.248	14
3968	Row7807#15	0.218	0.248	15
4224	Row8319#16	0.328	0.248	16
127	Row126#0	0.591	0.246	0
383	Row638#1	0.395	0.246	1
639	Row1150#2	0.646	0.246	2

Figure 8: 16 iterations of batch of 512 (i.e. total: 8192). The above table gives frequency-amplitude observed in each iteration for frequency of value=0.248.

Node9:

Calculate mean and variance of *power* (amplitude) over all iterations after grouping by frequency.

Rows: 256 | Columns: 3 Sorting Filter

#	RowID	freq Number (Float)	Mean(power) Number (Float)	Variance(power) Number (Float)
1	Row0	-0.25	0.374	0.051
2	Row1	-0.248	0.544	0.103
3	Row2	-0.246	0.572	0.154
4	Row3	-0.244	0.664	0.048
5	Row4	-0.242	0.918	0.112
6	Row5	-0.24	0.846	0.13
7	Row6	-0.238	1.03	0.257

Lower arm

Here we input a batch of 518 pts of defective bearing. Transform it using FFT. For every frequency we calculate '*power*' or amplitude.

Node 10:

Bad bearing data is input. Emitted data will remain bad till bearing is replaced.

Rows: 8970 | Columns: 7 Sorting Filter Search

#	RowID	sequence Number (Inte...	Acceler... Number ...	...	Accelerom... Number (Inte...	Accelerom... Number (Inte...	GyroscopeX Number (Inte...	GyroscopeY Number (Inte...	Gyroscope Number (Int
1	Row0	0	-131		-974	48	-9	3	13
2	Row1	1	149		-963	30	-11	6	4
3	Row2	2	140		-1008	40	-13	22	24
4	Row3	3	-107		-977	-6	-12	38	44

Node11:

Filter out only *AccelerometerX* data

Rows: 8970 Sorting Filter

#	RowID	AccelerometerX Number (Integer)
1	Row0	-131
2	Row1	149
3	Row2	140
4	Row3	-107

Node 12:

Just take top 512 points.

Rows: 512 | Sorting | Filter

#	RowID	AccelerometerX (Number (Integer))
1	Row0	-131
2	Row1	149
3	Row2	140

Node 13:

FFT output:

Rows: 512 | Columns: 3 | Sorting | Filter

#	RowID	power (Number (Float))	freq (Number (Float))	data (Number (Integer))
1	Row0	5.27	0	-131
2	Row1	2.249	0.002	149
3	Row2	0.059	0.004	140
4	Row3	0.093	0.006	-107
5	Row4	0.046	0.008	-179

Node 14:

Remove ‘data’ column

Node 15:

Filter missing values

Rows: 256 | Columns: 2 | Sorting | Filter

#	RowID	power (Number (Float))	freq (Number (Float))
1	Row0	5.27	0
2	Row1	2.249	0.002
3	Row2	0.059	0.004
4	Row3	0.093	0.006
5	Row4	0.046	0.008

Node 16

Join the top data with bottom data on 'frequency'.

Rows: 256 | Columns: 4 | Sorting | Filter

#	RowID	freq (Number (Float))	Mean(power) (Number (Float))	Variance(power) (Number (Float))	power (Number (Float))
1	Row0...	-0.25	0.374	0.051	2.407
2	Row1...	-0.248	0.544	0.103	1.147
3	Row2...	-0.246	0.572	0.154	3.979
4	Row3...	-0.244	0.664	0.048	4.354
5	Row4...	-0.242	0.918	0.112	3.695

Node 17

Standardise '*power*' column' and call it '*delta*'. Standardise as:

```
delta = Row wise Std power Value = (power - Mean(power))/Sqrt(Variance(power))
```

Here, per row:

Power: Last column value (say: 2.407)
Mean(power): Mean(power) column value (say: 0.374)
Variance(power): IInd last column (say: 0.051).

The last column '*delta*' below contains this per-row-value.

Rows: 256 | Columns: 5 | Sorting | Filter

#	RowID	freq (Number (Float))	Mean(power) (Number (Float))	Variance(power) (Number (Float))	power (Number (...)	delta (Number (Float))
1	Row0...	-0.25	0.374	0.051	2.407	9.005
2	Row1...	-0.248	0.544	0.103	1.147	1.879
3	Row2...	-0.246	0.572	0.154	3.979	8.693
4	Row3...	-0.244	0.664	0.048	4.354	16.831
5	Row4...	-0.242	0.918	0.112	3.695	8.317
6	Row5...	-0.24	0.846	0.13	1.163	0.879
7	Row6...	-0.238	1.03	0.257	0.815	0.425

Node 18:

Take a mean of '*delta*' column.

Rows: 1 | Sorting | Filter

#	RowID	delta Number (Float)
1	Row0	14.461

Node19:

IF it is greater than 2, call the COMPLETE BATCH ‘anomaly.’

Rows: 1 | Columns: 2 | Sorting | Filter

#	RowID	delta Number (Float)	predictior String
1	Row0	14.461	Anomaly!

Figure 9: Complete batch is anomalous.

Would freq of normal and bad data be the same?

In our problem, they are. But no.

No, **you do not assume they will have the same frequencies.** In fact, the entire power of using the FFT for anomaly detection relies on the expectation that normal data and bad data will look completely different in the frequency domain.

When a system transitions from a normal state to an anomalous (bad) state, its frequency signature changes in one of three ways:

1. New, Unexpected Frequencies Appear (Most Common)

Physical defects often introduce brand-new vibration or electrical frequencies that simply do not exist when the machine is running smoothly.

- **Normal Data:** A cooling fan rotates cleanly. The FFT shows only one major spike at **10 Hz** (the rotation speed).
- **Bad Data (Anomaly):** A bearing wears out, causing a rapid, microscopic rattle. The FFT suddenly shows the healthy **10 Hz** spike, plus a brand-new, high-frequency spike at **120 Hz**.

2. Expected Frequencies Shift in Amplitude (Spike or Drop)

Sometimes, the frequencies stay the same, but the amount of energy (power) at those frequencies shifts drastically.

- **Normal Data:** A structural beam vibrates slightly due to ambient wind, showing a tiny power level of **0.2** at **5 Hz**.
- **Bad Data (Anomaly):** The beam develops a structural crack, losing its stiffness. The same wind now causes it to violent shake, pushing the power at **5 Hz** up to **8.5**.

3. Frequencies Broaden into "Noise"

When components completely fail or break loose, organized, periodic frequencies break down into chaotic, non-periodic energy.

- **Normal Data:** Clean, sharp spikes at specific frequencies (e.g., a pure musical tone or steady hum).
- **Bad Data (Anomaly):** The sharp spikes flatten out, and the entire baseline of the graph lifts up, indicating white noise (like the static sound of a broken speaker).

🧠 **How the Algorithm Handles This**

Because normal and bad data have different frequency profiles, your anomaly detection pipeline will typically use one of two strategies:

- **The Threshold Method:** You teach the algorithm what **normal** looks like, and you build a "ceiling" (threshold) across the spectrum. If *any* frequency—whether it's an existing one or a brand-new one—crosses that ceiling, it flags an anomaly.
- **The Machine Learning Method:** You train a model (like an Isolation Forest or a neural network) on the FFT output. The model treats the power values of your 512 frequency bins as a unique "fingerprint" and easily learns to tell the healthy fingerprint apart from the broken one.

Why do I always get discrete frequency output in FFT?

Yes, that's exactly how FFT works. Here's why?
FFT gives you discrete frequency bins, not a continuous spectrum

When you feed the FFT algorithm a signal with N samples taken at a sampling rate fs, it doesn't return "the frequency content at any frequency you want." It returns exactly N (or N/2, if you only keep the non-negative-frequency half) discrete frequency bins, spaced evenly from 0 up to the Nyquist frequency (fs/2).

The spacing between consecutive frequency bins is:

$$\Delta f = fs / N$$

So in your table, the fact that freq goes 0, 0.002, 0.004, 0.006, 0.008, 0.01, 0.012… with a constant step of 0.002 is a direct fingerprint of this. That step size *is* your Δf, which tells you:

$$fs / N = 0.002$$

So if you know either your sampling rate or your number of samples, you can back out the other. For example, if N = 512 (matching your row count of 512), then fs = 0.002 × 512 ≈ 1.024.

Why this happens (intuition, not just math)

The FFT is really just evaluating the Discrete Fourier Transform, which assumes your finite chunk of N samples represents one period of a periodic signal. Because of that assumption, the only frequencies that "fit" cleanly into that window are integer multiples of 1/(N·Δt) — i.e., integer multiples of fs/N. Anything in between doesn't correspond to a whole number of cycles across your sample window, so it isn't a valid basis frequency for that particular DFT.

Practical implications

- **Frequency resolution** is fixed by Δf = fs/N — want finer resolution (smaller Δf)? You need more samples (longer time record), not a fancier FFT.

- **Zero-padding** (appending zeros to your signal before the FFT) will give you more points and interpolate between the "true" bins, making the plot look smoother — but it does NOT add real new information/resolution. It only makes the existing spectral shape easier to visualize.
- If your actual signal has a frequency component that falls between two bins (e.g., at 0.017 when your bins are 0.016 and 0.018), you'll see energy "leak" into the surrounding bins rather than one clean spike — this is called **spectral leakage**.

Finance data

data: C:\Users\ashok\OneDrive\Documents\anomaly_detection\FFT\finance_data
Source Claude desktop
Knime file: 'Fourier Transform for Anamoly Detection--Finance Data'
Prompt:

I have uploaded two csv files. Data in the files has been used in the Anomaly Detection process by Knime. The pdf article which describes the process is also attached. KNIME uses Fast Fourier Transforms for making anomaly prediction. I want a similar time-series data pertaining to finance.

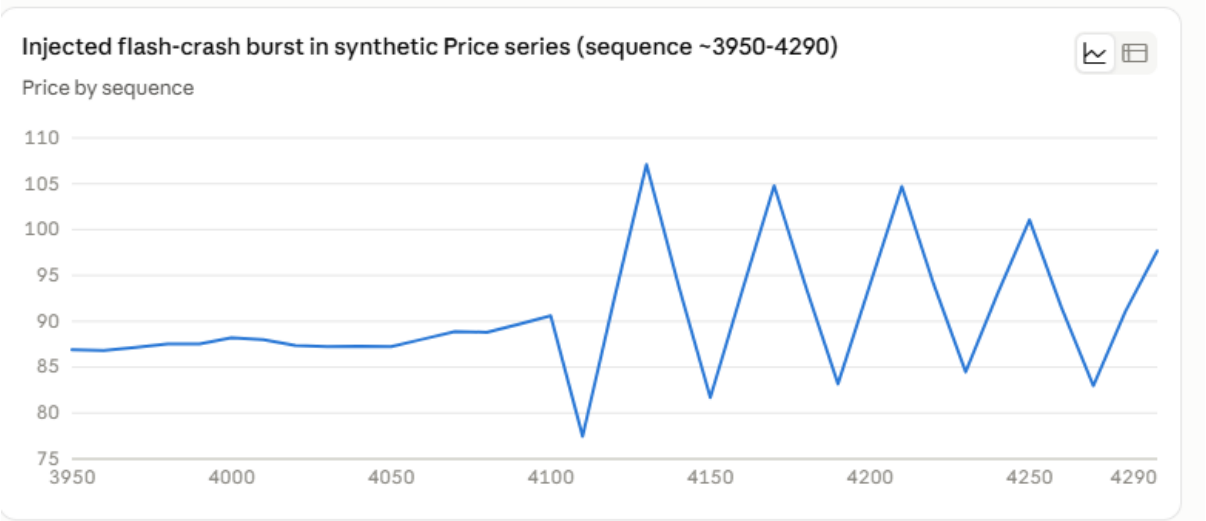


Figure 10: From point 3950 onwards data is anomalous. First, the data values are almost flat (and hence anomalous) and next they burst (and hence again anomalous). But note thisis anomaly is occurring only point 3950 onwards and not earlier.

I generated two finance-flavored CSVs that mirror the structure of your sensor files, so you can plug them into the same KNIME FFT anomaly-detection workflow.

Design (mirrors the turbine example):

- 6 numeric columns per row, like AccelerometerX/Y/Z + GyroscopeX/Y/Z → here it's **Price, Return, Volume, Volatility, BidAskSpread, OrderFlowImbalance**
- 8,192 rows = 16 × 512, so it slices cleanly into FFT windows of 512 (same window size the article uses)
- **non-anomalous_finance_data.csv** — a "normal" trading regime: price random-walk plus repeating intraday cycles (a 512-tick cycle for volume/volatility, shorter cycles layered on top, like the turbine's steady vibration signature)
- **anomalous_finance_data.csv** — identical generative process, but rows 4096–4607 (exactly one 512-window) get a flash-crash-style shock: a decaying high-frequency oscillation injected into every column, the same way the article injects a new vibration frequency for the "bad bearing" data

That shock shows up clearly as a burst of high-frequency energy the FFT wouldn't see in the normal windows:
The chart above shows the injected burst around sequence 4090–4290 — prices whipsaw at a much higher frequency than anything in the normal data, which is exactly the kind of signature the FFT + z-score approach in the article is built to catch.

To reuse the KNIME workflow as-is:

1. Point the two CSV Reader nodes at these files instead of the turbine ones
2. Pick one column at a time (e.g., Price) for the "one example column" filter, since the workflow processes a single series through FFT
3. Keep window size 512 — the anomalous window (rows 4096–4607) should land in exactly one Chunk Loop iteration and score highly

##########